

# Principios SOLID: Diseño de Software de Calidad

---

Nombre del autor

 **Tiempo estimado: 10 min**

Universidad San Carlos de Guatemala  
Facultad de Ingeniería  
Escuela de Ingeniería en Ciencias y Sistemas.  
Segundo Semestre 2026  
[Laboratorio - Nombre del Curso - Sección]

# 1. Marco Formativo

## 1.1 Valores

| Nombre del valor | ¿Cómo se aplica en tu laboratorio?                                                                                                                                                                                                                                                                      |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad  | Se evidencia al escribir código limpio, probado y mantenible. Un desarrollador responsable no entrega código sin haberlo validado. Aplicar SOLID es una forma de responsabilidad técnica: facilita el mantenimiento, las pruebas y reduce el riesgo de romper funcionalidad existente al hacer cambios. |

## 1.2 Competencia(s)

| Tipo de Competencia    |                                                                                                                        |
|------------------------|------------------------------------------------------------------------------------------------------------------------|
| Competencia General    | Implementa pruebas unitarias para validar componentes de software, garantizando que el código funciona como se espera. |
| Competencia Específica | Aplica los principios SOLID para mejorar la calidad del código, facilitando su mantenimiento, extensión y prueba.      |

# 2. Palabras Clave

**SOLID** : Acrónimo de cinco principios de diseño orientado a objetos que favorecen la mantenibilidad y la testabilidad del código.

**Acoplamiento** : Grado de dependencia entre componentes de un sistema. Un alto acoplamiento dificulta el mantenimiento.

**Abstracción** : Mecanismo que permite trabajar con conceptos generales sin depender de implementaciones específicas.

**Refactorización** : Proceso de mejorar la estructura interna del código sin cambiar su comportamiento externo.

### 3. Introducción

Escribir código que funciona no es lo mismo que escribir buen código. El software que no aplica principios de diseño sólidos tiende a volverse frágil con el tiempo: un cambio en una parte rompe otra, las pruebas son difíciles de escribir y agregar nuevas funcionalidades se convierte en un riesgo. Los principios SOLID, formulados por Robert C. Martin, ofrecen una guía para construir sistemas más robustos, mantenibles y testeables.

### 4. Contenidos

Presentación de temas y subtemas.

#### Los 5 Principios SOLID

---

##### **S — Single Responsibility Principle (Responsabilidad Única)**

---

"Una clase debe tener una sola razón para cambiar."

Cada clase debe encargarse de una única responsabilidad bien definida. Cuando una clase hace demasiadas cosas, un cambio en una funcionalidad puede romper otras de forma inesperada, y las pruebas unitarias se vuelven complicadas de escribir.

##### **O — Open/Closed Principle (Abierto/Cerrado)**

---

"Las clases deben estar abiertas a la extensión y cerradas a la modificación."

Debe ser posible agregar nuevas funcionalidades sin modificar el código que ya funciona correctamente. Esto generalmente se logra mediante interfaces y clases abstractas, que permiten extender el comportamiento sin tocar la implementación existente.



## **L — Liskov Substitution Principle (Sustitución de Liskov)**

---

"Las subclases deben poder reemplazar a sus clases base sin alterar el comportamiento del programa."

Si una clase B hereda de A, cualquier parte del sistema que espere un objeto de tipo A debe funcionar correctamente si recibe un objeto de tipo B. Violar este principio produce jerarquías de herencia que engañan al compilador pero fallan en tiempo de ejecución.

## **I — Interface Segregation Principle (Segregación de Interfaces)**

---

"Es mejor tener muchas interfaces específicas que una interfaz de propósito general."

No se debe obligar a una clase a implementar métodos que no necesita. Las interfaces deben diseñarse para ser pequeñas y precisas, de modo que cada clase implemente solo lo que realmente usa.

## **D — Dependency Inversion Principle (Inversión de Dependencias)**

---

"Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones."

La lógica del negocio no debe depender directamente de implementaciones concretas como bases de datos, APIs externas o archivos. Al depender de abstracciones, es posible cambiar la implementación sin afectar la lógica central y se facilita la escritura de pruebas con mocks.

**Tabla 1 — Resumen de los Principios SOLID**

| Letra | Principio                 | Pregunta clave                                                      |
|-------|---------------------------|---------------------------------------------------------------------|
| S     | Responsabilidad Única     | ¿Esta clase tiene una sola razón para cambiar?                      |
| O     | Abierto/Cerrado           | ¿Puedo agregar comportamiento sin modificar código existente?       |
| L     | Sustitución de Liskov     | ¿Cualquier subclase puede reemplazar a su clase base sin problemas? |
| I     | Segregación de Interfaces | ¿Las interfaces son específicas y sin métodos innecesarios?         |
| D     | Inversión de Dependencias | ¿Las clases reciben sus dependencias desde afuera?                  |

**Tabla 1 - Resumen de los Principios SOLID**

### Caso de estudio

Un equipo desarrolló un módulo de facturación que funcionaba correctamente. Con el tiempo, el sistema creció y se pidió agregar soporte para facturas en dólares además de quetzales. Al intentar hacer el cambio, el desarrollador encontró que la lógica de conversión de moneda, el cálculo de impuestos, la generación del PDF y el envío por correo estaban todos dentro de una misma clase de 800 líneas.

Modificar la conversión de moneda rompió el cálculo de impuestos. Arreglar los impuestos afectó la generación del PDF. El equipo tardó dos semanas en un cambio que debía tomar dos días, y durante ese tiempo introdujeron tres bugs nuevos.

Al analizar el problema, identificaron múltiples violaciones a SOLID: la clase tenía cinco responsabilidades distintas (violación de SRP), no era posible agregar nuevas monedas sin

modificar el código existente (violación de OCP) y todos los módulos dependían de implementaciones concretas en lugar de abstracciones (violación de DIP).

Tras refactorizar aplicando SOLID, el módulo quedó dividido en clases especializadas. Cuando semanas después se pidió agregar soporte para euros, el cambio tomó cuatro horas y no afectó ninguna otra funcionalidad.

## 5. Conclusiones

Los principios SOLID no son reglas absolutas, sino guías que deben aplicarse con criterio según el contexto del proyecto. No tiene sentido sobre-aplicarlos en scripts simples o prototipos.

El código que respeta SOLID es más fácil de probar unitariamente, lo que se traduce en mayor confianza al hacer cambios.

La responsabilidad profesional de un desarrollador incluye no solo hacer que el código funcione, sino asegurarse de que pueda mantenerse y evolucionar sin convertirse en una deuda técnica.

## 6. Referencias

1. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
2. Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.